TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 | **SECTION:** | CPE21S4 |
| **DATE PERFORMED:** | 7/29/26 | **DATE SUBMITTED:** | 7/29/26 |
| **NAME:** | Rayven Ferry Licas | **INSTRUCTOR:** | ENGR. Jimlord Quejado |

## ACTIVITY NO. ___ [ACTIVITY NAME]

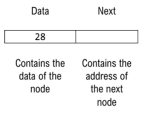
## 1. PROCEDURE OUTPUT

What procedures did you perform?

**ILO A: Construct C++ code for a singly and doubly linked list in C++**

**A.1. Singly Linked List**

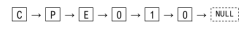
To start, we will do a simple implementation of a linked list. We must keep in mind the visual representation of the individual nodes that will make up our linked list:




Every node in a singly linked list will have 2 compartments, the data and the pointer to the next. The data contains the element of a single type that we want it to contain. The link will point to the next node in our list. In C++, the node is defined as:

```
class Node{
public:
    char data;
    Node *next;
};
```

We will implement a list to represent the string "CPE010" that looks like the figure below:




Implementation will follow the given steps:

1. Create the node pointers and initialize as NULL
2. Create new instances of the node class and allocate them in the heap
3. Define every node in the list
4. Point the last node to null

Simple implementation:

```
#include<iostream>
#include<utility>

class Node{
public:
    char data;
    Node *next;
};

int main(){
    //step 1
    Node *head = NULL;
```

19

```
    Node *second = NULL;
    Node *third = NULL;
    Node *fourth = NULL;
    Node *fifth = NULL;
    Node *last = NULL;

    //step 2
    head = new Node;
    second = new Node;
    third = new Node;
    fourth = new Node;
```

Although we have created the linked list, this is an implementation that is useless and meant only to show what we want to happen for the given output. **Run your code and screenshot the output, then briefly discuss how the output came to be and how could it be improved (table 3-1)?**

Imagine if we had to make multiple items in the list in the hundreds or thousands! This would be too tedious, ineffective, and inefficient. So, we must implement certain methods. These methods/operations associated with the linked lists are:

- Traversal
- Insertion at head
- Insertion at any part of the list
- Insertion at the end
- Deletion of a node

Linked List Traversal

```
    Algorithm: ListTraversal (parameter: pointer to node n)
    WHILE n IS NOT EQUAL TO null
        PRINT data OF n
        GO TO NEXT NODE n := next
    ENDWHILE
```

20

```
    PRINT next line
    END
```

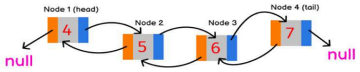
Sometimes we have a linked list, and we need to insert a node somewhere other than at the end of the list. We will look at a couple of different ways to insert a node into an existing list.

To insert a node at the head:

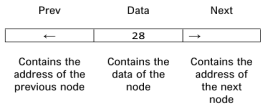
1. Allocate memory for the new node

**A.2. Doubly Linked List**

The singly linked list allows for direct access from a list node only to the next node in the list. A doubly linked list allows convenient access from a list node to the next node and also to the preceding node on the list.




The doubly linked list node accomplishes this in the obvious way by storing two pointers: one to the node following it (as in the singly linked list), and a second pointer to the node preceding it.




This means that in addition to our implementation of the singly linked list, we'll have addition compartment.

```
class Node{
public:
    char data;
    Node *next;
    Node *prev;
};
```

**Modify the given operations used in the singly linked lists to work on the new construct of a doubly linked list. Provide a screenshot of your code and analysis in table 3-4.**

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.



```cpp
#include <iostream>
using namespace std;

class Node {
public:
    char data;
    Node* next;
};

void display(Node* head) {
    Node* temp = head;

    while (temp != NULL) {
        cout << temp->data;
        temp = temp->next;
    }

    cout << endl;
}

void insertHead(Node*& head, char value) {
    Node* newNode = new Node;

    newNode->data = value;
    newNode->next = head;

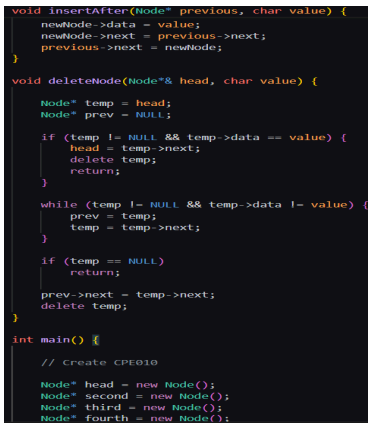
    head = newNode;
}

void insertAfter(Node* previous, char value) {

    if (previous == NULL)
        return;

    Node* newNode = new Node;

    newNode->data = value;
```



```cpp
void insertAfter(Node* previous, char value) {
    newNode->data = value;
    newNode->next = previous->next;
    previous->next = newNode;
}

void deleteNode(Node*& head, char value) {

    Node* temp = head;
    Node* prev = NULL;

    if (temp != NULL && temp->data == value) {
        head = temp->next;
        delete temp;
        return;
    }

    while (temp != NULL && temp->data != value) {
        prev = temp;
        temp = temp->next;
    }

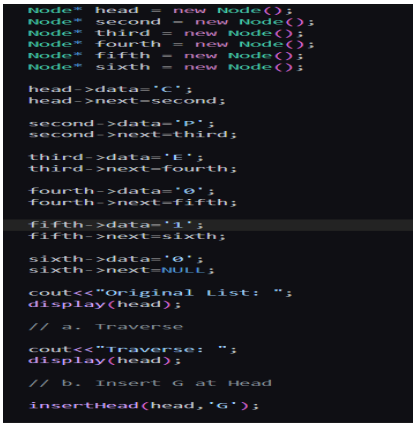
    if (temp == NULL)
        return;

    prev->next = temp->next;
    delete temp;
}

int main() {

    // Create CPE010

    Node* head = new Node();
    Node* second = new Node();
    Node* third = new Node();
    Node* fourth = new Node();
```



```cpp
Node* head = new Node();
Node* second = new Node();
Node* third = new Node();
Node* fourth = new Node();
Node* fifth = new Node();
Node* sixth = new Node();

head->data='C';
head->next=second;

second->data='P';
second->next=third;

third->data='E';
third->next=fourth;

fourth->data='0';
fourth->next=fifth;

fifth->data='1';
fifth->next=sixth;

sixth->data='0';
sixth->next=NULL;

cout<<"Original List: ";
display(head);

// a. Traverse

cout<<"Traverse: ";
display(head);

// b. Insert G at Head

insertHead(head,'G');
```



```cpp
insertHead(head,'G');

cout<<"After inserting G at head: ";
display(head);

// c. Insert E after P

Node* temp=head;

while(temp!=NULL){

    if(temp->data=='P'){
        insertAfter(temp,'E');
        break;
    }

    temp=temp->next;
}

cout<<"After inserting E after P: ";
display(head);

// d. Delete C

deleteNode(head,'C');

cout<<"After deleting C: ";
display(head);

// e. Delete P

deleteNode(head,'P');

cout<<"After deleting P: ";
display(head);

// f. Final Output
```

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.